NagaEd.com
+91 9366807251
hr@nagaed.com
3rd Floor DVK Building, Sanyü-o Gudzü
New Secretariat Road, Nagaland 797001
CIN: U80904NL2020PTC013550

Hurdle Task: Software Developer

Hurdle Task — Mid-Level

Total Duration	Approximately 4–5 hours (manage your time effectively)
Submission	Submit all 4 tasks via the online form within 3 days of receiving this task
Stack	React, Node.js (Express), FastAPI (Python), MySQL, Docker
Format	Source code as a ZIP or GitHub link + one-page architecture diagram (PNG/PDF)

Objective: This task reflects the real day-to-day work at NagaEd.

You will build a small but complete full-stack feature: a **user registration and authentication system** with a React frontend, a Node.js REST API, a FastAPI AI microservice, and a Docker Compose setup that ties it all together.

Read all four tasks before you begin - they are interdependent.

Submission Instructions:

- Push your code to a public GitHub repository, OR zip the entire project directory.
- Include a top-level README.md with your name, contact details, and full setup instructions.
- Include your architecture diagram as a PNG or PDF in the repository root or zip.
- Submit via https://forms.gle/5nXtuDKGrgqUCnx1A with subject line: [Your Full Name] — Software Developer Hurdle Task Submission

Important

Do **not** submit only screenshots, a PDF explanation, or a Word document.

The application itself must be included through the **GitHub repository or ZIP file** so that the hiring team can review and run the implementation.

NagaEd.com
+91 9366807251
hr@nagaed.com
3rd Floor DVK Building, Sanyü-o Gudzü
New Secretariat Road, Nagaland 797001
CIN: U80904NL2020PTC013550

TASK 1 – Backend API with Node.js

Estimated Time: 1.5–2 hours

Build a REST API in Node.js (Express or Fastify) that handles user registration and authentication. This is the core backend service all other tasks depend on.

Instructions:

1. Initialise a Node.js project and create a REST API with the following endpoints:
    - POST /api/register — accepts { username, email, password, address } in the request body.
    - POST /api/login — accepts { email, password } and returns a signed JWT on success.
    - GET /api/users/:id — protected route; returns the user record (excluding password) when a valid JWT is provided.
2. Store users in a MySQL database. Hash passwords using bcrypt before storing.
3. Validate all incoming fields and return appropriate 4xx errors for missing or malformed data.
4. Include a README that explains how to install dependencies and run the server.

What Good Looks Like:

- Clean separation of concerns (routes, controllers, services, DB layer).
- Sensible HTTP status codes and consistent error response shape.
- No plain-text passwords anywhere - ever.

NagaEd.com
+91 9366807251
hr@nagaed.com
3rd Floor DVK Building, Sanyü-o Gudzü
New Secretariat Road, Nagaland 797001
CIN: U80904NL2020PTC013550

TASK 2 – React Frontend

Estimated Time: 1–1.5 hours

Build a minimal React interface that consumes the API from Task 1.

This tests your ability to connect a modern frontend to a REST backend and handle real-world UI concerns such as loading states and errors.

Instructions:

5. Create a React app (Vite or Create React App) with two pages or views:
    - Register
    - Login
6. The Register form should collect:
    - Username
    - Email
    - Password
    - Address
7. The Login form should collect:
    - Email
    - Password
8. On successful login, store the returned JWT and display the logged-in user's username on a simple dashboard page.
9. Handle and display errors returned from the API, for example:
    - "Email already in use"
    - "Invalid credentials"
10. The UI does not need to be polished, but it must be functional and not broken on a standard desktop viewport.

What Good Looks Like:

- API calls extracted into a service or hook layer, not inline in components.
- Loading and error states handled gracefully.
- No hardcoded credentials or API keys in the code.

NagaEd.com
+91 9366807251
hr@nagaed.com
3rd Floor DVK Building, Sanyü-o Gudzü
New Secretariat Road, Nagaland 797001
CIN: U80904NL2020PTC013550

TASK 3 – FastAPI AI Microservice

Estimated Time: 1–1.5 hours

Build a small Python FastAPI service that adds an AI-powered feature to the application.
This reflects the kind of work done on Marina AI at NagaEd.

Instructions:

10. Create a FastAPI service (separate from the Node.js API) with a single endpoint: POST /api/suggest-username
11. The endpoint accepts: { interests: string[] } in the request body and uses the **Groq API** to call an LLM and suggest **3 unique, creative usernames** based on the provided interests.
12. Return the suggestions as a JSON object:

{ suggestions: ["name1", "name2", "name3"] }

13. Add error handling for cases where:
- The Groq API call fails.
- The input is malformed.
14. The FastAPI /docs (Swagger UI) page should render correctly and be usable without additional setup.

Note:

Sign up for a free Groq API key at console.groq.com. No credit card is required.

Store the key as an environment variable: GROQ_API_KEY

Never hardcode the API key.

What Good Looks Like:

- Pydantic models are used for request and response validation.
- The LLM API key is loaded from an environment variable and never hardcoded.
- The prompt is thoughtful and gives the model clear, concise instructions.
- The /docs page renders correctly and describes the endpoint accurately.

NagaEd.com
+91 9366807251
hr@nagaed.com
3rd Floor DVK Building, Sanyü-o Gudzü
New Secretariat Road, Nagaland 797001
CIN: U80904NL2020PTC013550

TASK 4 – Docker Compose

Estimated Time: 1 hour

Containerise all three services and wire them together with Docker Compose.

This reflects how NagaEd deploys applications on AWS.

Instructions:

15. Write a Dockerfile for the Node.js API and a separate Dockerfile for the FastAPI service. The React app can be served from the Node.js container or a simple Nginx container.
16. Write a docker-compose.yml that starts all services together. Services must communicate by **container service name, not localhost**.
17. Environment variables, including:

- JWT secret
- Groq API key
- Database credentials must be loaded via a .env file.

Do not hardcode them in Dockerfiles or the compose file.

18. Running: docker compose up should bring up the entire application with no manual steps beyond copying .env.example to .env.
19. Include a brief architecture diagram showing:
    - Services
    - Ports
    - How the services communicate

What Good Looks Like:

- Multi-stage Docker builds to keep image sizes small.
- .dockerignore files present in each service directory.
- Health checks defined in the compose file.
- Containers run as a non-root user.
- Only necessary ports are exposed publicly.

NagaEd.com
+91 9366807251
hr@nagaed.com
3rd Floor DVK Building, Sanyü-o Gudzü
New Secretariat Road, Nagaland 797001
CIN: U80904NL2020PTC013550

Evaluation Criteria

We evaluate submissions holistically.

You will **not be penalised for using documentation or AI tools** - we do that every day.

What matters is the **quality of your thinking and the cleanliness of your implementation.**

Area	What We're Looking For
Code Quality	Readable, maintainable code with a sensible structure. We should be able to onboard your codebase easily.
Full-Stack Depth	Clear understanding of how the frontend, backend, and microservice interact end-to-end.
Security Basics	Passwords hashed, JWTs validated, secrets in environment variables, and no obvious injection vectors.
Container Knowledge	Docker setup works correctly with docker compose up. Services communicate correctly.
AI Integration	The FastAPI microservice is functional, the Groq integration works, and the prompt is well-crafted.
Communication	README is clear and accurate. Code is well-structured and easy to follow.

Good luck - we look forward to seeing what you build.